

TP : OpenGL 4 - Premier programme

Jonathan Fabrizio
<http://jo.fabrizio.free.fr/>

Objectif

L'objectif de ce T.P. est d'écrire un premier programme en utilisant OpenGL 4.

1 Travail préliminaire

Pour pouvoir réaliser notre premier programme nous devons disposer d'un certain nombre de fonctions pour manipuler les matrices et gérer les transformations dans l'espace. Il y a donc des fonctions générales (produit matriciel...) mais aussi des fonctions plus spécifiques à la 3D (pour initialiser une matrice projection par exemple). Dans la pratique, vous pouvez soit programmer vous même ces fonctions, soit utiliser une bibliothèque (comme glm) qui fait le travail pour vous. Ici vous allez devoir les écrire vous rapidement. Pour la partie spécifique, vous devez programmer les fonctions `look_at` et `frustum` qui permettent respectivement de positionner la caméra et de faire la projection. Vous pourrez par la suite, si besoin, ajouter d'autres fonctions (pour gérer les rotations et translations par exemple).

1.1 Matrices et transformations

1.1.1 Matrices

Écrire une classe `matrix4` contenant (au moins mais vous pourrez compléter au besoin) les méthodes et constructeur suivants :

```
matrix4();  
void operator*=(const matrix4& rhs);  
static matrix4 identity();
```

Ajoutez une fonction pour écrire une matrice sur un flux :

```
std::ostream& operator<<(std::ostream &out, const mygl::matrix4 &m);
```

Les éléments seront enregistrés avec des `GLfloat`. Si ces méthodes ne sont pas suffisantes, vous pouvez ajouter ce que vous voulez et vous pouvez aussi ajouter une classe `vec4` sur le même principe.

1.1.2 Transformations

Écrire les fonctions suivantes `look_at` et `frustum` qui construisent une matrice afin de gérer respectivement la position/l'orientation de la caméra et la projection.

On pourra s'aider des anciennes spécifications d'OpenGL pour les écrire :

- <https://www.khronos.org/registry/OpenGL-Refpages/gl2.1/xhtml/glFrustum.xml>
- <https://www.khronos.org/registry/OpenGL-Refpages/gl2.1/xhtml/gluLookAt.xml>

`look_at`

```
... look_at(  
    const GLfloat &eyeX, const GLfloat &eyeY, const GLfloat &eyeZ,  
    const GLfloat &centerX, const GLfloat &centerY, const GLfloat &centerZ,  
    const GLfloat &upX, const GLfloat &upY, const GLfloat &upZ  
);
```

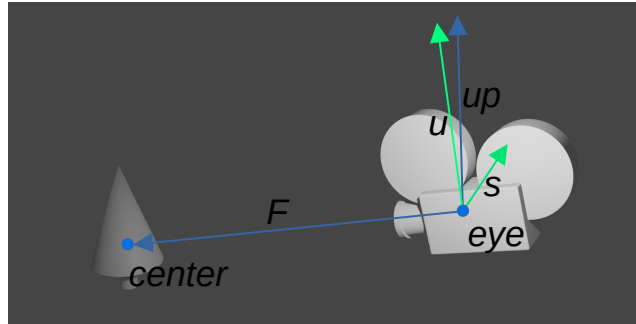
L'objectif de cette fonction est de calculer la matrice qui permettra le changement de repère pour passer du repère scène au repère caméra. Ici

- (`eyeX`, `eyeY`, `eyeZ`) représente le centre de la caméra,

- (centerX, centerY, centerZ) le point regardé, et
 - (upX, upY, upZ) un vecteur qui indique la direction vers le *haut* de la caméra.
- On peut en déduire la direction vers laquelle la caméra est orientée $\vec{F}$:

$$\vec{F} = \begin{pmatrix} center_X - eye_X \\ center_Y - eye_Y \\ center_Z - eye_Z \end{pmatrix} \quad (1)$$

Pensez à normaliser $\vec{F}$. On doit aussi corriger le vecteur qui indique le haut de la caméra, le vecteur $\vec{up}$ n'ayant peut être pas exactement la bonne orientation. Pour cela, on calcule le vecteur à l'aide du produit vectoriel : $\vec{s} = \vec{F} \times \vec{up}$. On peut maintenant déduire de $\vec{s}$ la direction $\vec{u}$ du haut de la caméra : $\vec{u} = \vec{s} \times \vec{F}$. On a maintenant bien $\vec{u}$ orthogonal à $\vec{F}$ et on a un repère, centré sur (centerX, centerY, centerZ) avec la base $(\vec{s}, \vec{u}, \vec{F})$. L'image suivante illustre les notations.



Le changement de repère à faire est donc donné par la matrice L :

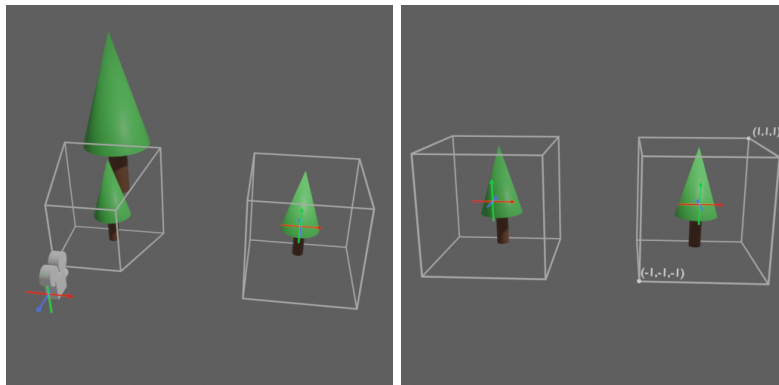
$$L = \begin{pmatrix} s_x & s_y & s_z & 0 \\ u_x & u_y & u_z & 0 \\ -F_x & -F_y & -F_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2)$$

Cette matrice combine le changement de repère que l'on vient de calculer avec la translation qui permet de centrer la scène sur la caméra. L'objectif de la fonction `look_at` est donc de construire cette matrice.

frustum

```
... frustum(
    const GLfloat &left, const GLfloat &right,
    const GLfloat &bottom, const GLfloat &top,
    const GLfloat &z_near, const GLfloat &z_far
);
```

L'objectif de cette fonction est de calculer le changement de repère qui permet au final d'envoyer tous les points visibles de la caméra dans les coordonnées normalisées d'OpenGL (ou presque) c'est à dire le cube $(-1, -1, -1), (1, 1, 1)$. Tous les autres points doivent être à l'extérieur de ce cube afin d'être éliminés. Dans la pratique, on va calculer une matrice M . Pour tout point $P = (p_x, p_y, p_z, 1)$, exprimé en coordonnées homogènes dans le repère caméra, donnera un point $P' = (p'_x, p'_y, p'_z, p'_w) = MP$. Le point $(p'_x/p'_w, p'_y/p'_w, p'_z/p'_w, 1)$ devra être dans le cube pour être conservé.



2 Programme OpenGL

Entrons dans le vif du sujet. Votre fonction main va ressembler à :

```
...
init_glut(argc, argv);
init_glew();
init_GL();
init_shaders();
init_object();
init_POV();
glutMainLoop();
...
```

Avec en plus la gestion des erreurs lorsqu'une fonction d'initialisation échoue.

2.1 Initialisations

On va devoir implémenter toutes les fonctions *bool init....()* invoquées dans le `main()`.

2.1.1 GLUT

Voilà un point de départ pour `init_glut()` mais vous devez le compléter au besoin :

```
bool init_glut(int &argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitContextVersion(4, 5);
    glutInitContextProfile(GLUT_CORE_PROFILE);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE | GLUT_DEPTH);
    glutInitWindowSize(1024, 1024);
    glutInitWindowPosition(10, 10);
    glutCreateWindow("Test OpenGL - POGL");
    ...
    return true;
}
```

2.1.2 GLEW

Idem pour `init_glew()` :

```
bool init_glew() {
    return (glewInit() == GLEW_OK)
}
```

2.1.3 GL

A vous de jouer pour `init_gl()` ! Vous n'avez peut être pas d'idée pour l'instant mais vous pouvez compléter au fur et à mesure (couleur de fond, *backface culling*, *z-buffer*...

```
bool init_gl() {
    ...
}
```

2.1.4 Compilation des Shaders

Écrire une classe *program* qui permet de gérer les *shaders* et les programmes. Elle devra comporter au moins les méthodes et constructeur/destructeur suivants :

```
program::program()
program::~~program()
static program *make_program(std::string &vertex_shader_src, std::string &fragment_shader_src)
char *get_log();
bool is_ready();
void use();
```

À la construction du programme, à l'appel de `make_program (factory)`, il faut compiler les deux *shaders* séparément (`glCreateShader()` + `glShaderSource()` + `glCompileShader()` + `glGetShader()` + `glGetShaderInfoLog()`) puis invoquer l'éditeur de liens (`glCreateProgram()` + `glAttachShader()` + `glglLinkProgram()` + `glGetProgram()` + `glGetProgramInfoLog()`). `get_log()` renvoie les logs pour la compilation ou l'édition de liens. `is_ready()` indique si un programme est prêt à être utilisé (i.e. si la compilation et l'édition de liens ont réussi). `use()` active l'utilisation de ce programme (`glUseProgram()`) pour le rendu. Une fois l'édition de lien réussi, on pourra détruire les *shaders* pour économiser un peu de place.

Une fois fait, vous pouvez implémenter `init_shaders()`. Vous devrez écrire deux petits *shaders*, (simple pour l'instant, projection+couleur unie).

Écriture des *shaders*

Vous devez maintenant écrire le *vertex shader* et le *fragment shader*. Vous aurez :

- un *uniform* pour la couleur de l'objet,
- un *uniform* pour la matrice qui fait passer du repère monde à la caméra,
- un *uniform* pour la matrice projection.

En entrée du *vertex shader* vous aurez :

- un VBO pour les sommets de l'objet à dessiner,

Avec ces *shaders*, l'objet sera d'une couleur unie. Il n'aura donc pas d'effet de volume.

A l'aide de votre classe *program*, compilez vos *shaders* et corrigez les éventuelles erreurs.

2.1.5 Initialisation d'un objet

Il faut maintenant écrire `init_object()`. Il faut créer un VBO avec les positions de vos sommets puis le lier au variables *in* de votre *vertex shader*. Corriger *vertex shader* au besoin. Tout cela sans oublier le VAO.

Vous vous dites peut être quel objet ? Je ne sais pas un cube, une pyramide, un lapin (original?!)... peu importe. L'important, c'est que l'objet soit bien en face de la camera, et entre le *near plane* et le *far plane* et pas trop gros pour ne pas déborder à droite à gauche, en haut, en bas. Cela implique aussi de bien penser à la position de la camera, son orientation ainsi que son *frustum*.

2.1.6 Initialiser le point de vue

Il va falloir écrire `init_POV()` pour initialiser les matrices *model-view* et *projection*. Il faut initialiser ces matrices dans les *shaders*, donc les déclarer *uniform* et fixer leur valeur (`glUniform()`).

2.2 Le coeur du programme

On approche du but ! On n'a même jamais été aussi près !

2.2.1 La fonction *display()*

Écrivez votre fonction `display()`. Elle devra : effacer l'image et le *z-buffer* (`glClear()`), activer le bon VAO, le bon *shader* (le bon *program* plutôt), fixer la couleur de l'objet (vous avez un *uniform* pour cela puis faire enfin le *draw* (`glDrawArrays()`) ! Sans oublier d'échanger les *buffers* (Ha oui le *double buffering*).

Pensez à dire à GLUT que pour l'affichage, c'est votre fonction *display()*, qu'il faut invoquer (cf. `glutDisplayFunc()`).

2.2.2 *Go ! go ! go !*

Invoquer `glutMainLoop()` et Tadam !

Vous pouvez lancer votre programme, constater qu'il ne fait exactement ce que vous vouliez qu'il fasse et du coup repasser toutes les étapes tranquillement pour trouver et corriger le (les) problèmes.

Je vous propose, pour vous aider, que vous alliez récupérer ma macro dans le squelette pour tester la présence d'erreur après l'exécution d'une commande OpenGL. Vous ne pouvez pas ne pas tester la présence d'une erreur après n'importe quelle primitive OpenGL.

2.2.3 Ça marche!!!!

Maintenant que votre programme fonctionne (Youpi!), faites un *TAG* sur votre dernier *commit* pour pouvoir le retrouver au cas où vous auriez tout cassé (Oui car dès le début vous avez initialisé un dépôt).

2.2.4 Pour aller plus loin...

Maintenant que votre programme fonctionne (Youpi!), faites un *TAG* sur votre dernier *commit* pour pouvoir le retrouver au cas ou vous auriez tout cassé (Oui car dès le debut vous avez bien évidemment initialisé un depot git) `git`).

Lorsque vous aurez terminé votre T.P., vous pourrez ajouter des fonctionnalités tel que :

- le lissage de l'objet.

Pour cela vous devrez peut etre :

- ajoutez un *VBO* pour les normales

- ajoutez un *uniform* pour la position de la lumière et un autre pour la couleur/l'intensité de la lumière.

Modifiez le vertex shader pour prendre en compte ces modifications.

Pour finir, et comme il vous reste encore un peu de temps, amusez vous avec vos *shaders* pour faire des effets sympas...

3 Après avoir fini et pour aller plus loin

3.1 Shaders

Vous pouvez modifier votre programme et imaginer plein d'améliorations comme :

- donner la possibilité de changer l'angle de vue de votre objet,

- modifier la couleur du rendu en fonction de la distance à l'observateur,

- ...